BDY1103 — TALLER DE BASE DE DATOS | DUOC UC (EVALUACIÓN PARCIAL N° 1)

Guía de Defensa Oral y Explicación de Código — Sonido Vivo

Proyecto: Sonido Vivo (Viña del Mar)	**Docente Evaluador:** Josue Oteiza
Integrantes: Dylan Sandoval, Victor Martinez, Danilo Sepúlveda	**Estado del Repo:** Sincronizado en main (Scripts 01 al 05)

DIAPOSITIVA 1: Portada e Identificación

APERTURA

■■ *QUÉ DECIR EN LA PRESENTACIÓN:*

"Buenas tardes, profesor Josue Oteiza y compañeros. Junto a mi equipo conformado por Dylan Sandoval, Victor Martinez y Danilo Sepúlveda, venimos a presentar la solución que desarrollamos en base de datos para la tienda de instrumentos musicales 'Sonido Vivo', ubicada en Viña del Mar. Nuestro objetivo fue transformar la gestión manual de la tienda en un sistema seguro, ordenado y automático usando directamente las capacidades de Oracle."

DIAPOSITIVA 2: Contexto de Negocio y Problemática

CRITERIO A

■■ *QUÉ DECIR EN LA PRESENTACIÓN:*

"Sonido Vivo vende guitarras, teclados y equipos de audio en Viña del Mar. Llevaban todo a mano en cuadernos y planillas, con dos riesgos gravísimos: vender productos sin stock físico en bodega y la imposibilidad de aplicar descuentos automáticos por compras grandes. Con nuestra solución en Oracle PL/SQL, cada venta descuenta stock real y los descuentos se calculan solos en memoria."

DIAPOSITIVA 3: Modelo Entidad-Relación y Reglas

SCRIPT 01

■■ *QUÉ DECIR EN LA PRESENTACIÓN:*

"Diseñamos la base de datos completamente normalizada: las categorías agrupan productos, los clientes compran y cada venta se divide en su detalle. Pero lo más potente son los candados que pusimos mediante restricciones CHECK: es imposible tener precios o stocks negativos, y ninguna boleta se guarda si el total no cuadra exactamente con subtotal menos descuento."

CÓDIGO DEMOSTRATIVO:

```
-- Candado matemático en VENTA
CONSTRAINT chk_venta_totales CHECK (total = (subtotal - descuento))

-- Candados físicos y de cálculo en DETALLE_VENTA
CONSTRAINT chk_det_cant CHECK (cantidad > 0),
CONSTRAINT chk_det_subtotal CHECK (subtotal_linea = cantidad * precio_unitario)
```

EXPLICACIÓN PASO A PASO:

- **chk_venta_totales:** Obliga a nivel de kernel a que el total sea matemáticamente exacto; cancela cualquier inserción errónea.
- **chk_det_cant:** Prohíbe cantidades nulas o negativas (no se puede comprar 0 o menos guitarras).
- **chk_det_subtotal:** Valida en el motor que subtotal sea cantidad multiplicada por el precio unitario.

DIAPOSITIVA 4: Datos que Procesa el Sistema

CRITERIO B

■■ *QUÉ DECIR EN LA PRESENTACIÓN:*

"Por un lado, el sistema recibe el catálogo con sus stocks críticos, los clientes con su nivel de fidelidad y las boletas emitidas en caja. Y con eso, nuestro motor calcula en memoria las tasas de descuento, consolida los balances en LIQUIDACION_CLIENTE y audita cualquier incidente en LOG_ERRORES."

DIAPOSITIVA 5: Descuentos con RECORD y VARRAY

SCRIPT 02

■■ QUÉ DECIR EN LA PRESENTACIÓN:

"Para liquidar sin poner lento el servidor, usamos dos herramientas en memoria: un RECORD como cajita donde agrupamos cliente, total bruto, descuento y neto en una sola variable; y un VARRAY como lista fija con los descuentos de 0%, 5% y 10%. Al tener los porcentajes a mano en la memoria PGA, no hacemos SELECTs repetitivos al disco por cada cliente, haciendo el proceso veloz."

CÓDIGO DEMOSTRATIVO:

```
TYPE t_rec_liquidacion IS RECORD (
    id_cliente CLIENTE.id_cliente%TYPE, total_bruto NUMBER(10),
    tasa_descuento_pct NUMBER(5,2), total_neto NUMBER(10)
);
TYPE t_arr_tasas IS VARRAY(3) OF NUMBER(5,2);
v_escalas_descuento t_arr_tasas := t_arr_tasas(0.00, 5.00, 10.00);

IF v_liquidacion.total_bruto < 200000 THEN
    v_liquidacion.tasa_descuento_pct := v_escalas_descuento(1); -- 0%
ELSIF v_liquidacion.total_bruto BETWEEN 200000 AND 700000 THEN
    v_liquidacion.tasa_descuento_pct := v_escalas_descuento(2); -- 5%
ELSE
    v_liquidacion.tasa_descuento_pct := v_escalas_descuento(3); -- 10%
END IF;
```

EXPLICACIÓN PASO A PASO:

- **RECORD:** Agrupa atributos heterogéneos actuando como DTO; el %TYPE asegura compatibilidad con la tabla.
- **VARRAY(3):** Colección fija en memoria PGA que almacena las tasas base de descuento.
- **v_escalas_descuento(i):** Recupera la tasa en microsegundos sin hacer consultas a disco ni cambios de contexto SQL/PLSQL.

DIAPOSITIVA 6: Cursores Explícitos y Bucles Anidados

SCRIPT 03

■■ QUÉ DECIR EN LA PRESENTACIÓN:

"Para revisar las compras usamos dos cursores en equipo: el cursor padre busca a los clientes con compras activas; y por cada uno, llama al cursor hijo pasándole el ID del cliente como parámetro. El cursor hijo consulta únicamente las compras de esa persona, evitando mezclar datos y mostrando un reporte ordenado boleta por boleta."

CÓDIGO DEMOSTRATIVO:

```
-- Cursor Padre: Clientes con ventas
CURSOR c_clientes IS
    SELECT id_cliente, nombres || ' ' || apellidos AS cliente_nombre
    FROM CLIENTE c WHERE EXISTS (SELECT 1 FROM VENTA v WHERE v.id_cliente = c.id_cliente);

-- Cursor Hijo Parametrizado: Detalle de ese cliente específico
CURSOR c_ventas_cliente (p_id_cli CLIENTE.id_cliente%TYPE) IS
    SELECT v.nro_boleta, p.nombre, d.cantidad, d.subtotal_linea
    FROM VENTA v JOIN DETALLE_VENTA d ON v.id_venta = d.id_venta
    JOIN PRODUCTO p ON d.id_producto = p.id_producto
    WHERE v.id_cliente = p_id_cli;
```

EXPLICACIÓN PASO A PASO:

- **WHERE EXISTS:** Optimiza la búsqueda deteniendo el escaneo apenas confirma la primera compra en el índice.
- **(p_id_cli ...):** Parámetro formal que recibe el ID del cliente actual del bucle padre.
- **WHERE v.id_cliente = p_id_cli:** El optimizador filtra en la fuente usando índices, sin cargar productos cartesianos en memoria.

DIAPOSITIVA 7: Control de Excepciones y Auditoría

SCRIPT 04

■■ QUÉ DECIR EN LA PRESENTACIÓN:

"El control de errores es el freno de mano. Atajamos errores de Oracle como NO_DATA_FOUND si un producto no existe, y errores propios de negocio como e_stock_insuficiente si piden más de lo que queda en bodega. Ante una falla, el sistema aplica un ROLLBACK para deshacer la venta a medias y anota el incidente en LOG_ERRORES para no interrumpir la operación."

CÓDIGO DEMOSTRATIVO:

```
IF v_cantidad_solicitada > v_stock_actual THEN
    RAISE e_stock_insuficiente; -- Freno de mano
END IF;
...
EXCEPTION
    WHEN e_stock_insuficiente THEN
        ROLLBACK; -- Deshace para no corromper la BD
        INSERT INTO LOG_ERRORES (id_log, codigo_error, mensaje_error, usuario_bd, fecha_registro)
        VALUES (SEQ_LOG_ERRORES.NEXTVAL, 'USR-001', 'Stock insuficiente para ' || v_nombre_prod, USER, SYSDATE);
        COMMIT;
```

EXPLICACIÓN PASO A PASO:

- **RAISE e_stock_insuficiente:** Interrumpe el flujo normal y salta inmediatamente a la sección EXCEPTION.
- **ROLLBACK:** Garantiza atomicidad ACID anulando cualquier modificación transaccional incompleta.
- **SEQ_LOG_ERRORES.NEXTVAL:** Genera la clave primaria de auditoría de forma atómica en memoria compartida (SGA).

DIAPOSITIVA 8: Arquitectura Futura: Triggers Previstos

SCRIPT 05

■■ QUÉ DECIR EN LA PRESENTACIÓN:

"Pensando en la siguiente etapa y en cómo este motor se va a conectar con la página web de Fullstack II, la rúbrica nos pedía plantear la arquitectura futura. Nosotros no quisimos dejarlo solamente como una idea en el papel: ya dejamos prevista, diseñada y construida de antemano la lógica de los triggers en el script 05 del repositorio, dejándolos listos para cuando el negocio pase a producción: Primero, el trigger TRG_DESCUENTA_STOCK, para que al entrar una venta el stock en bodega baje automáticamente; y segundo, el trigger TRG_AUDITA_STOCK_CRITICO, que vigila la bodega y, si un producto queda en 3 unidades o menos, registra de inmediato una alerta en LOG_ERRORES. Además, están diseñados sobre tablas separadas para evitar el clásico error de tabla mutante (ORA-04091)."

CÓDIGO DEMOSTRATIVO:

```
-- Trigger 1 previsto: Rebaja automática al vender
CREATE OR REPLACE TRIGGER TRG_DESCUENTA_STOCK
AFTER INSERT ON DETALLE_VENTA
FOR EACH ROW
BEGIN
    UPDATE PRODUCTO SET stock = stock - :NEW.cantidad WHERE id_producto = :NEW.id_producto;
END;
/

-- Trigger 2 previsto: Alerta preventiva de bodega
CREATE OR REPLACE TRIGGER TRG_AUDITA_STOCK_CRITICO
AFTER UPDATE OF stock ON PRODUCTO FOR EACH ROW
WHEN (NEW.stock <= NEW.stock_critico)
BEGIN
    INSERT INTO LOG_ERRORES (id_log, codigo_error, mensaje_error, usuario_bd, fecha_registro)
    VALUES (SEQ_LOG_ERRORES.NEXTVAL, 'WARN-STOCK-CRITICO', 'ALERTA REPOSICIÓN: ' || :NEW.codigo, USER, SYSDATE);
END;
/
```

EXPLICACIÓN PASO A PASO:

- **AFTER INSERT ON DETALLE_VENTA:** Descuenta el stock físico tras la confirmación atómica de cada producto.
- **WHEN (NEW.stock <= NEW.stock_critico):** Filtro inteligente; solo alerta si el stock cae en zona de peligro para reabastecimiento.
- **Mitigación ORA-04091:** DETALLE_VENTA modifica la tabla externa PRODUCTO; no hay riesgo de mutación de tabla.

DIAPOSITIVA 9: Conclusiones y Ronda de Preguntas

CIERRE

■■ *QUÉ DECIR EN LA PRESENTACIÓN:*

"En conclusión, profesor: cumplimos al 100% con los bloques anónimos de liquidación, cursores y excepciones que pedía la evaluación, y además dejamos la evolución futura totalmente planificada, con los triggers ya preparados de antemano en el repositorio para cuando conectemos la base de datos con la plataforma web. Muchas gracias por su tiempo, quedamos atentos a sus preguntas."

Banco de Preguntas Frecuentes del Docente (Q&A; de Emergencia)

P1: ¿Por qué usaron un VARRAY y no una tabla anidada (Nested Table)?

Respuesta: "Elegimos VARRAY porque las escalas de descuento son una colección fija, acotada (3 tramos) y de orden estricto conocido de antemano. El VARRAY es más compacto y se aloja contiguamente en la memoria rápida (PGA)."

P2: ¿Por qué parametrizar el cursor hijo y no hacer un solo JOIN masivo?

Respuesta: "Porque necesitábamos mostrar un reporte jerárquico ordenado por cliente. Parametrizar permite que el motor de Oracle filtre en cada vuelta usando el índice del cliente, manteniendo el uso de memoria bajo control."

P3: En LIQUIDACION_CLIENTE guardan nombre_completo, ¿eso no viola la 3FN respecto a CLIENTE?

Respuesta: "Técnicamente sí, pero se aplica deliberadamente el patrón de Snapshot Histórico (Cierre Contable): una liquidación es un balance inmutable en el tiempo. Si el cliente cambia su apellido en el futuro, no debemos alterar el balance contable histórico.

P4: ¿Por qué usan secuencias (SEQUENCE) y no MAX(id) + 1?

Respuesta: "Porque MAX(id) + 1 falla si dos ventas ocurren al mismo tiempo (colisión de clave primaria). Las secuencias de Oracle entregan identificadores de forma atómica en memoria compartida sin bloqueos."

P5: ¿Qué es una tabla mutante (ORA-04091) y cómo la previenen?

Respuesta: "Ocurre cuando un trigger intenta consultar o modificar la misma tabla que lo disparó. En nuestros triggers lo evitamos porque el trigger se dispara en DETALLE_VENTA y modifica la tabla externa PRODUCTO."

P6: ¿Tenemos triggers en esta entrega? (Respuesta con el encuadre previsor):

Respuesta: "En esta entrega la pauta pedía bloques anónimos. Pero nosotros dejamos prevista, diseñada y construida de antemano la lógica de los triggers en el script 05, para que cuando la tienda pase a producción, los disparadores ya estén listos para funcionar."